

TREES

Data Structures

Tree

A tree is recursively defined as a set of one or more nodes where one node is designated as the root of the tree and all the remaining nodes can be partitioned into non-empty sets each of which is a sub-tree of the root.

Types

General Tree: General trees are data structures that store elements hierarchically

Forest : disjoint union of trees.

Binary Tree : In a binary tree, the topmost element is called the root node, and each node has 0, 1, or at the most 2 children.

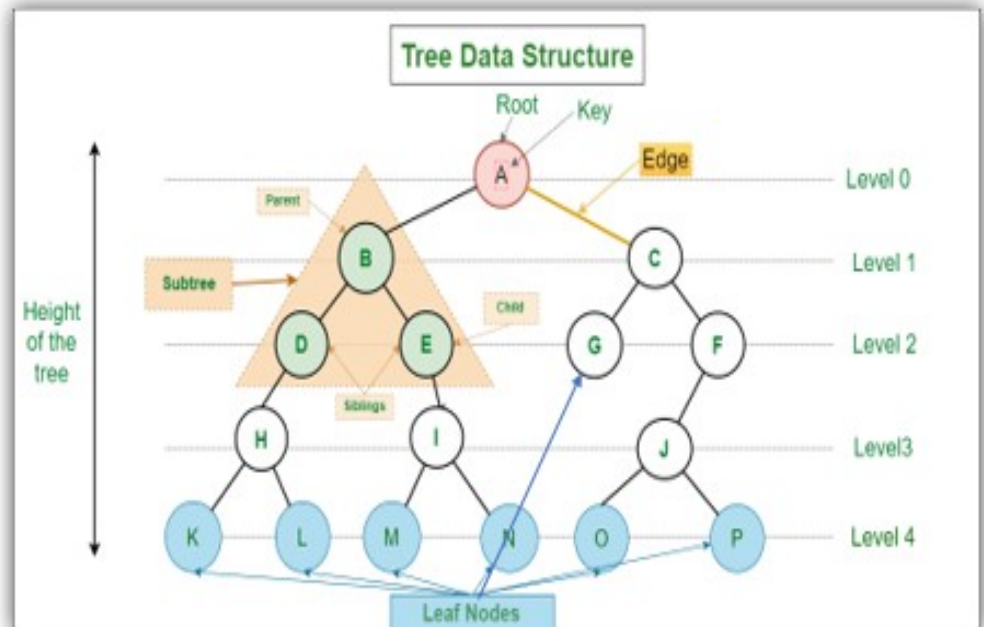
Binary Search Tree :

Expression Tree

Basic Terminologies In Tree Data Structure:

- Parent Node:** The node which is a predecessor of a node is called the parent node of that node. {B} is the parent node of {D, E}.
- Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: {D, E} are the child nodes of {B}.
- Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. {G, K, L, M, N, O, P} are the leaf nodes of the tree.
- Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. {A,B} are the ancestor nodes of the node {E}

- Descendant:** Any successor node on the path from the leaf node to that node. {E,I} are the descendants of the node {B}.
- Sibling:** Children of the same parent node are called siblings. {D,E} are called siblings.
- Level of a node:** The count of edges on the path from the root node to that node. The root node has level 0.
- Internal node:** A node with at least one child is called Internal Node.
- Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- Subtree:** Any node of the tree along with its descendant.



Subtree: If Root is not NULL then T_L and T_R are subtree.

Edge: A line drawn from node N to its successor

Path: a sequence of consecutive edges.

Branch: a path ending in a leaf

Depth The *depth* of a node N is given as the length of the path from the root R to the node N. The depth of the root node is zero.

Height of a tree It is the total number of nodes on the path from the root node to the deepest node in the tree. A tree with only a root node has a height of 1

A binary tree of height h has at least h nodes and at most $2^h - 1$ nodes.

The height of a binary tree with n nodes is at least $\log_2(n+1)$ and at most n.

Similar tree: Tree having structure or shape

Copies: if they are similar and they have same content at corresponding nodes

Calaton number = $C(2n, n) / (n+1) = 2n! / (n+1)!n!$

Gives how many different binary tree are constructed using n nodes.

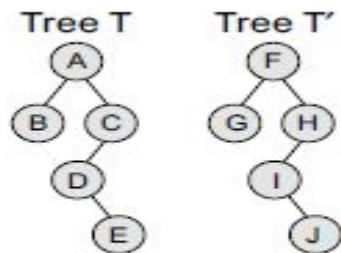


Figure 9.5 Similar binary trees

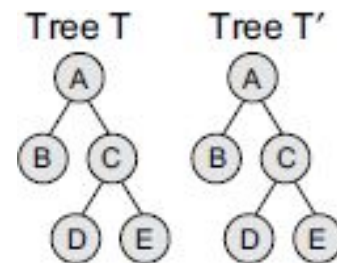


Figure 9.6 T' is a copy of T

Binary Tree

In a binary tree, the topmost element is called the root node, and each node has 0, 1, or at the most 2 children.

1. Full Binary Tree-

- A binary tree in which every node has either 0 or 2 children is called as a **Full binary tree**.
- Full binary tree is also called as **Strictly binary tree**.

2. Complete Binary Tree

Complete Binary Tree has all levels completely filled with nodes except the last level and in the last level, **all the nodes are as left side as possible**.

A complete binary tree is a binary tree in which node insertion takes place from the left.



✗ Full Binary Tree
✗ Complete Binary Tree



✓ Full Binary Tree
✗ Complete Binary Tree



✗ Full Binary Tree
✓ Complete Binary Tree

Properties of Complete Binary Tree

- level r of T can have at most 2^r nodes
- if K is a parent node, then its left child can be calculated as $2 \times K$ and its right child can be calculated as $2 \times K + 1$
- The parent of the node K can be calculated as $\lfloor K/2 \rfloor$
- The height of a tree T having exactly n nodes is given as: $H = \lfloor \log_2 (n + 1) \rfloor$

Extended Binary Tree

A binary tree T is said to be an extended binary tree (or a 2-tree) if each node in the tree has either no child or exactly two children. In an extended binary tree, nodes having two children are called *internal nodes* and nodes having no children are called *external nodes*. The internal nodes are represented using circles and the external nodes are represented using squares.

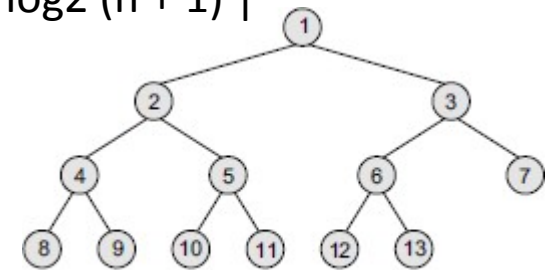


Figure 9.7 Complete binary tree

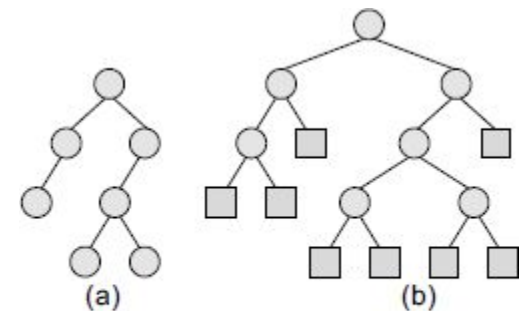


Figure 9.8 (a) Binary tree and (b) extended binary tree

Representation of Binary Trees in the Memory

Linked representation of binary trees

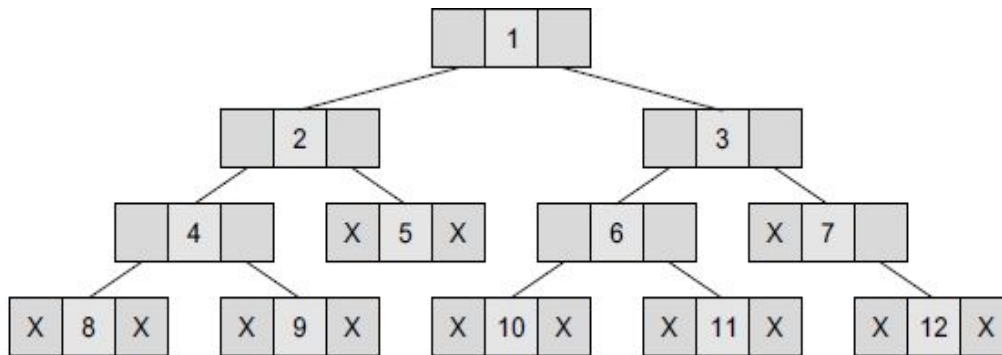


Figure 9.9 Linked representation of a binary tree

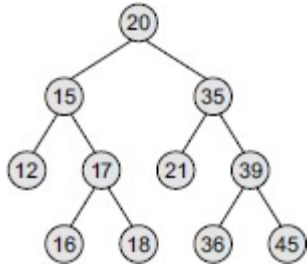
```

struct node {
    struct node *left;
    int data;
    struct node *right;
};
  
```

	LEFT	DATA	RIGHT
ROOT 3	1	8	-1
	2	10	-1
	3	5	1
	4		8
	5	9	2
	6		14
	7		
	8	20	3
	9	1	4
	10		11
	11	-1	7
	12	-1	9
	13		-1
	14	-1	5
	15		-1
	16	-1	11
	17		-1
	18	-1	12
	19		-1
	20	2	6

Figure 9.11 Linked representation of binary tree T

Sequential Representation/ Array Representation



1	20
2	15
3	35
4	12
5	17
6	21
7	39
8	
9	
10	16
11	18
12	
13	
14	36
15	45

- A one-dimensional array, called TREE, is used to store the elements of tree.
- The root of the tree will be stored in the first location. That is, TREE[1] will store the data of the root element.
- The children of a node stored in location K will be stored in locations $(2 \times K)$ and $(2 \times K+1)$.
- The maximum size of the array TREE is given as $(2^h - 1)$, where h is the height of the tree.
- An empty tree or sub-tree is specified using NULL. If TREE[1] = NULL, then the tree is empty.

Traversing a Binary Tree

Pre-order Traversal (NLR)

1. Visiting the root node,
2. Traversing the left sub-tree, and finally
3. Traversing the right sub-tree.

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:     Write TREE -> DATA
Step 3:     PREORDER(TREE -> LEFT)
Step 4:     PREORDER(TREE -> RIGHT)
           [END OF LOOP]
Step 5: END
```

Figure 9.16 Algorithm for pre-order traversal

In-order

1. Traversing the left sub-tree,
2. Visiting the root node, and finally
3. Traversing the right sub-tree.

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:     INORDER(TREE -> LEFT)
Step 3:     Write TREE -> DATA
Step 4:     INORDER(TREE -> RIGHT)
           [END OF LOOP]
Step 5: END
```

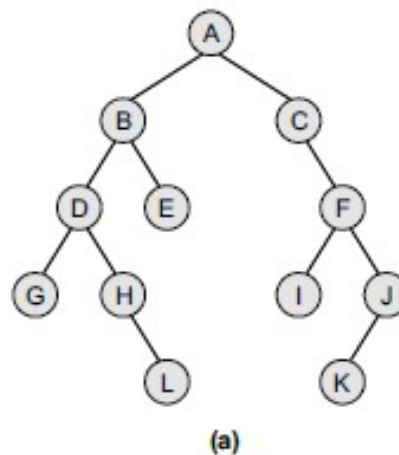
Figure 9.17 Algorithm for in-order traversal

Post-order Traversal (LRN)

1. Traversing the left sub-tree,
2. Traversing the right sub-tree, and finally
3. Visiting the root node.

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:     POSTORDER(TREE -> LEFT)
Step 3:     POSTORDER(TREE -> RIGHT)
Step 4:     Write TREE -> DATA
           [END OF LOOP]
Step 5: END
```

Figure 9.18 Algorithm for post-order traversal



A, B, D, G, H, L, E, C, F, I, J,
and K

G, D, H, L, B, E, A, C, I, F, K, and J

G, L, H, D, E, B, I, K, J, F, C, and A

```

#include <stdio.h>
#include <stdlib.h>
struct node
{
    int item;
    struct node *left;
    struct node *right;
};
// In-order traversal algorithm
void inOrder(struct node *root)
{
    if (root == NULL)
        return;
    inOrder(root->left);
    printf(" %d ", root->item);
    inOrder(root->right);
}
// pre-order Traversal algorithm
void preOrder(struct node *root)
{
    if (root == NULL)
        return;
    printf(" %d ", root->item);
    preOrder(root->left);
    preOrder(root->right);
}
// post-order traversal algorithm
void postOrder(struct node *root)
{
    if (root == NULL)
        return;
    postOrder(root->left);
    postOrder(root->right);
    printf(" %d ", root->item);
}

```

```

int main()
{
    struct node *root, *one, *two, *three, *four, *seven ;
    root=(struct node*)malloc(sizeof(struct node));
    one = (struct node*)malloc(sizeof(struct node));
    two = (struct node*)malloc(sizeof(struct node));
    three = (struct node*)malloc(sizeof(struct node));
    four = (struct node*)malloc(sizeof(struct node));
    seven=(struct node*)malloc(sizeof(struct node));
    root = one;
    one-> left= two;
    one->right= three;
    one->item=1;
    two->left= four;
    two->right=NULL;
    two->item=2;
    three->left=NULL;
    three->right=seven;
    three->item=3;
    four->left=NULL;
    four->right=NULL;
    four->item=4;
    seven->left=NULL;
    seven->right=NULL;
    seven->item = 7;
    printf("Preoder traversal\n");
    preOrder(root);
    printf("\nInoder traversal\n");
    inOrder(root);
    printf("\nPostorder traversal\n");
    postOrder(root);
    return 0;
}

```

```

Preoder traversal
1 2 4 3 7
Inoder traversal
4 2 1 3 7
Postorder traversal
4 2 7 3 1

```

Constructing Binary tree from Traversal

Step 1 Use the pre-order/ post-order sequence to determine the root node of the tree.

Step 2 Elements on the left side of the root node in the in-order traversal sequence form the left sub-tree of the root node. Similarly, elements on the right side of the root node in the in-order traversal sequence form the right sub-tree of the root node.

Step 3 Recursively select each element from pre-order/post-order traversal sequence and create its left and right sub-trees from the in-order traversal sequence.

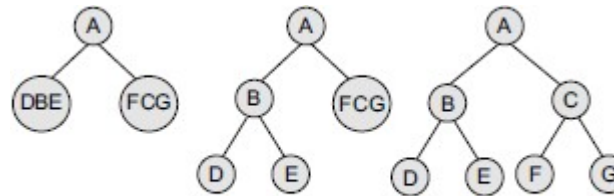


Figure 9.20

Pre-order Traversal: A B D E C F G

In-order Traversal: D B E A F C G

In-order Traversal: D B H E I A F J C G

Post order Traversal: D H I E B J F G C A

Binary Search Tree

A binary search tree is a binary tree with the following properties:

1. The left sub-tree of a node N contains values that are less than N 's value.
2. The right sub-tree of a node N contains values that are greater than N 's value.
3. Both the left and the right binary search trees also satisfy these properties and, thus, are binary search trees.

Operations

Insertion It takes $O(\log n)$ time to execute in the average case and $O(n)$ time in the worst case.

Searching can be done in $O(\log_2 n)$ time to execute in the average case and $O(n)$ time in the worst case.

`searchElement (TREE, VAL)`

Step 1: IF `TREE → DATA = VAL` OR `TREE = NULL`

 Return `TREE`

ELSE

 IF `VAL < TREE → DATA`

 Return `searchElement(TREE → LEFT, VAL)`

 ELSE

 Return `searchElement(TREE → RIGHT, VAL)`

 [END OF IF]

 [END OF IF]

Step 2: END

`Insert (TREE, VAL)`

Step 1: IF `TREE = NULL`

 Allocate memory for `TREE`

 SET `TREE → DATA = VAL`

 SET `TREE → LEFT = TREE → RIGHT = NULL`

ELSE

 IF `VAL < TREE → DATA`

 Insert(`TREE → LEFT, VAL`)

 ELSE

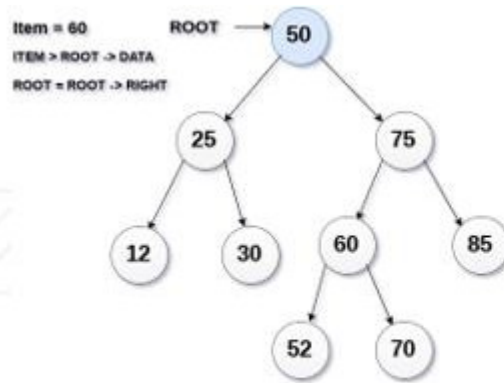
 Insert(`TREE → RIGHT, VAL`)

 [END OF IF]

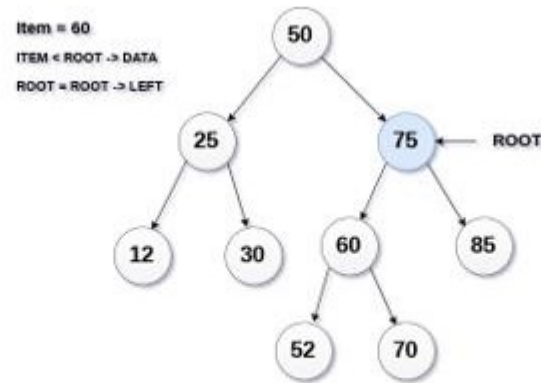
 [END OF IF]

Step 2: END

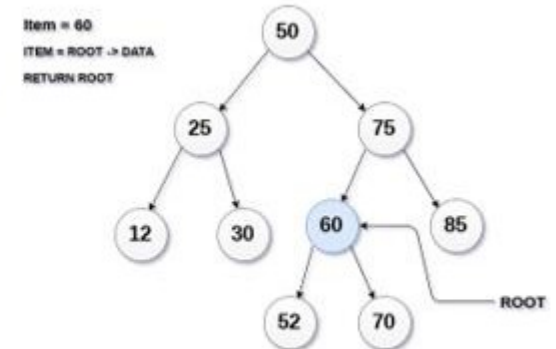
Searching in BST



STEP 1



STEP 2



STEP 3

Example 10.2 Create a binary search tree using the following data elements:

45, 39, 56, 12, 34, 78, 32, 10, 89, 54, 67, 81

Solution

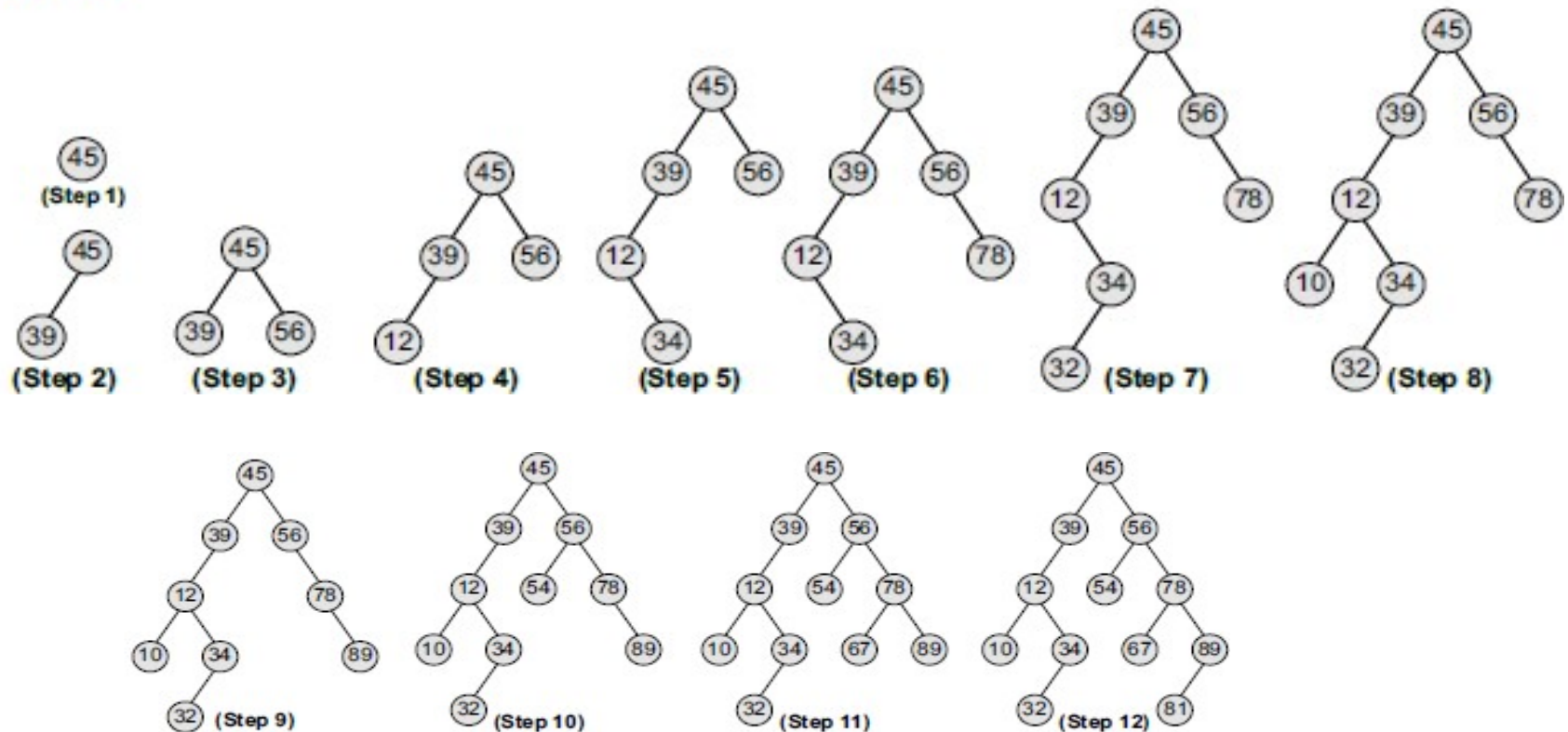


Figure 10.4 Binary search tree

Deletion from BST

Case 1: Deleting a Node that has No Children

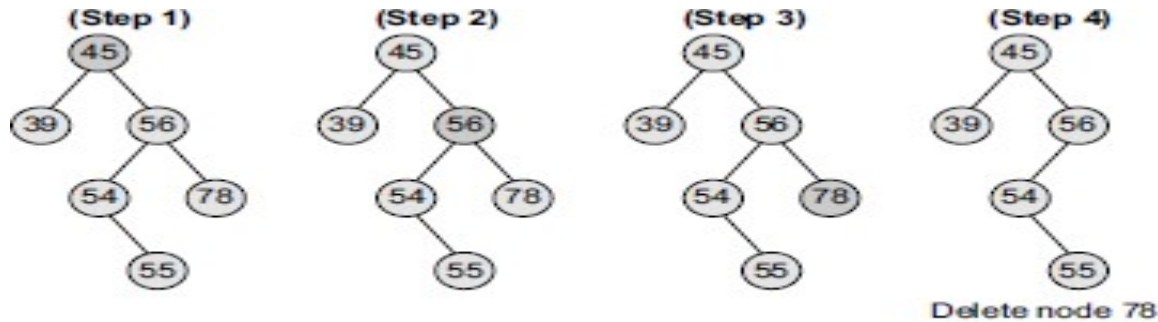


Figure 10.11 Deleting node 78 from the given binary search tree

Case 2: Deleting a Node with One Child : replace the node with its child.

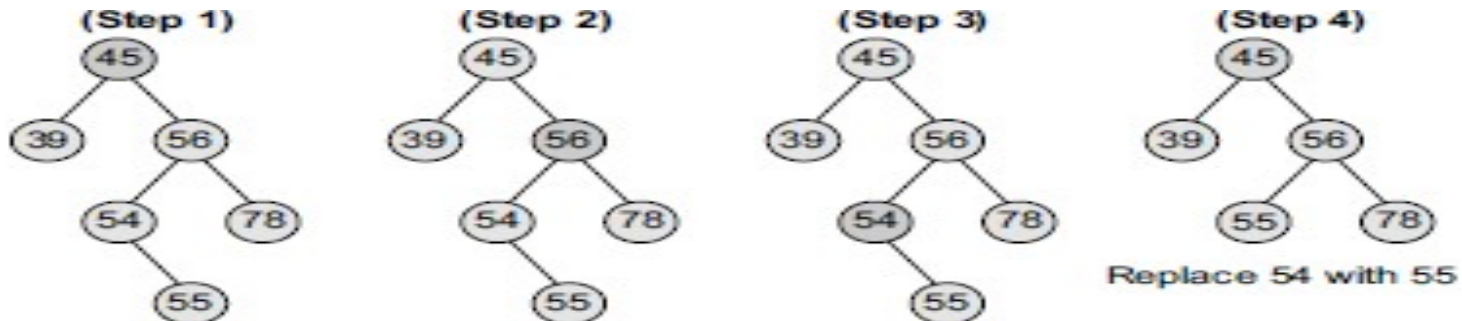


Figure 10.12 Deleting node 54 from the given binary search tree

Case 3: Deleting a Node with Two Children : replace the node's value with its *in-order predecessor* (largest value in the left sub-tree) or *in-order successor* (smallest value in the right sub-tree). The in- order predecessor or the successor can then be deleted using any of the above cases.

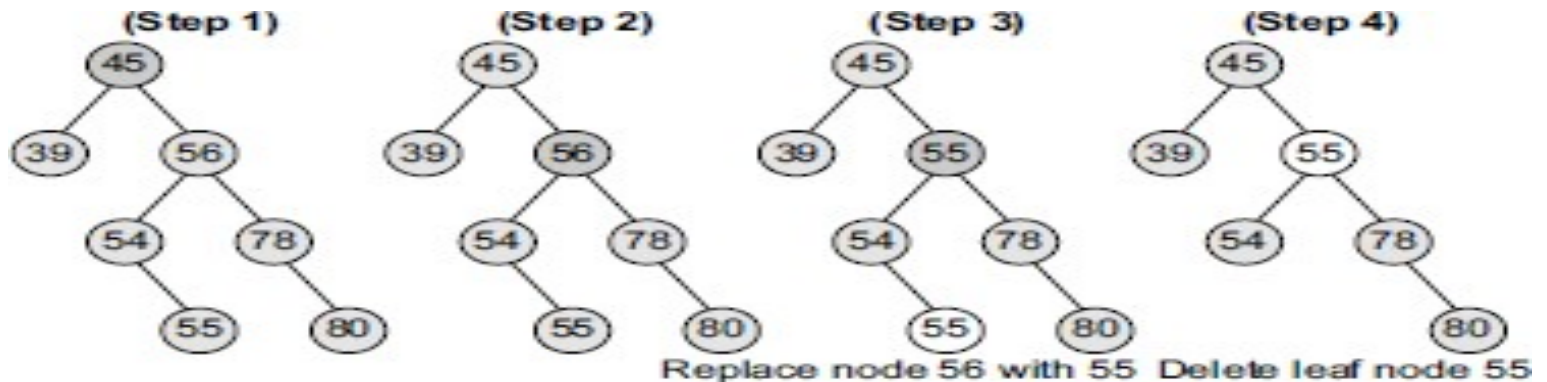


Figure 10.13 Deleting node 56 from the given binary search tree

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data; //node will store some data
    struct node *right_child; // right child
    struct node *left_child; // left child
};

//function to create a node
struct node* new_node(int x) {
    struct node *temp;
    temp = (struct node*) malloc(sizeof(struct node));
    temp -> data = x;
    temp -> left_child = NULL;
    temp -> right_child = NULL;
    return temp;
}

// searching operation
struct node* search(struct node * root, int x) {
    if (root == NULL || root -> data == x) //if root->data is x then
        the element is found
        return root;
    else if (x > root -> data) // x is greater, so we will search the
        right subtree
        return search(root -> right_child, x);
    else //x is smaller than the data, so we will search the left
        subtree
        return search(root -> left_child, x);
}
```

```
// insertion
struct node* insert(struct node * root, int x) {
    //searching for the place to insert
    if (root == NULL)
        return new_node(x);
    else if (x > root -> data) // x is greater. Should be inserted to
        the right
        root -> right_child = insert(root -> right_child, x);
    else // x is smaller and should be inserted to left
        root -> left_child = insert(root -> left_child, x);
    return root;
}
```

```
//function to find the minimum value in a node
struct node* find_minimum(struct node * root) {
    if (root == NULL)
        return NULL;
    else if (root -> left_child != NULL) // node with minimum value
        will have no left child
        return find_minimum(root -> left_child); // left most element
        will be minimum
    return root;
}
```

```
// deletion
struct node* delete(struct node * root, int x) {
    //searching for the item to be deleted
    if (root == NULL)
        return NULL;
    if (x > root -> data)
        root -> right_child = delete(root -> right_child, x);
    else if (x < root -> data)
        root -> left_child = delete(root -> left_child, x);
    else {
        //No Child node
        if (root -> left_child == NULL && root -> right_child == NULL)
        {
            free(root);
            return NULL;
        }
        //One Child node
        else if ((root -> left_child == NULL && root->left_child!=NULL)
            || (root -> right_child == NULL && root->left_child != NULL)
        ) {
            struct node *temp;
            if (root -> left_child == NULL )
                temp = root -> right_child;
            else
                temp = root -> left_child;
            free(root);
            return temp;
        }
    }
}
```

```
//Two Children
else {
    struct node *temp = find_minimum(root -> right_child);
    root -> data = temp -> data;
    root -> right_child = delete(root -> right_child, temp ->
        data);
    }
}
return root;
}

// Inorder Traversal
void inorder(struct node *root) {
    if (root != NULL) // checking if the root is not null
    {
        inorder(root -> left_child); // traversing left child
        printf(" %d ", root -> data); // printing data at root
        inorder(root -> right_child); // traversing right child
    }
}
```



```
int main() {
    struct node *root;
    root = new_node(20);
    insert(root, 5);
    insert(root, 1);
    insert(root, 15);
    insert(root, 9);
    insert(root, 7);
    insert(root, 12);
    insert(root, 30);
    insert(root, 25);
    insert(root, 40);
    insert(root, 45);
    insert(root, 42);

    inorder(root);
    printf("\n");
}
```

```
    root = delete(root, 1);
    inorder(root);
    printf("\n");

    root = delete(root, 40);
    inorder(root);
    printf("\n");

    root = delete(root, 45);
    inorder(root);
    printf("\n");

    return 0;
}
```

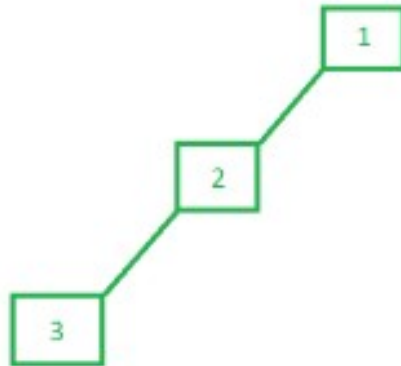
1	5	7	9	12	15	20	25	30	40	42	45
5	7	9	12	15	20	25	30	40	42	45	
5	7	9	12	15	20	25	30	42	45		
5	7	9	12	15	20	25	30	42			

Skewed Tree

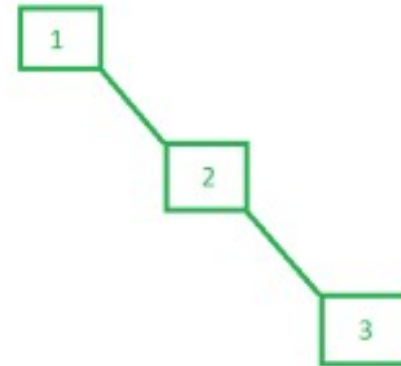
A skewed binary tree is a type of binary tree in which all the nodes have only either one child or no child.

Types of Skewed Binary trees

There are 2 special types of skewed tree:



LEFT SKEWED



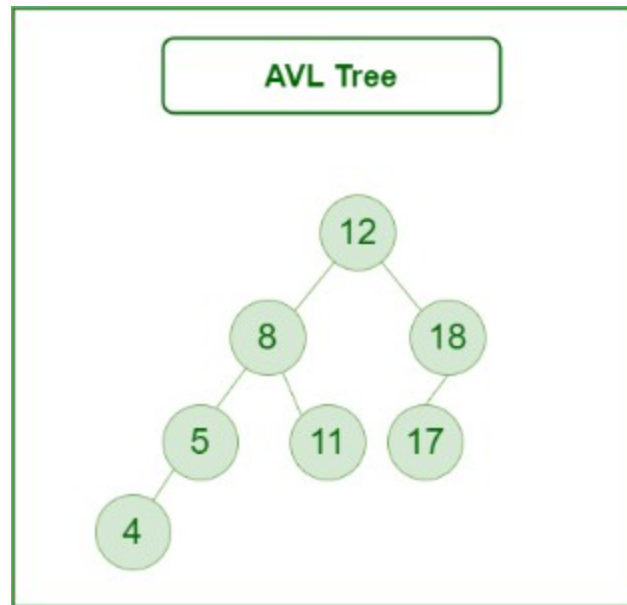
RIGHT SKEWED

AVL Tree

An **AVL tree** defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees for any node cannot be more than one.

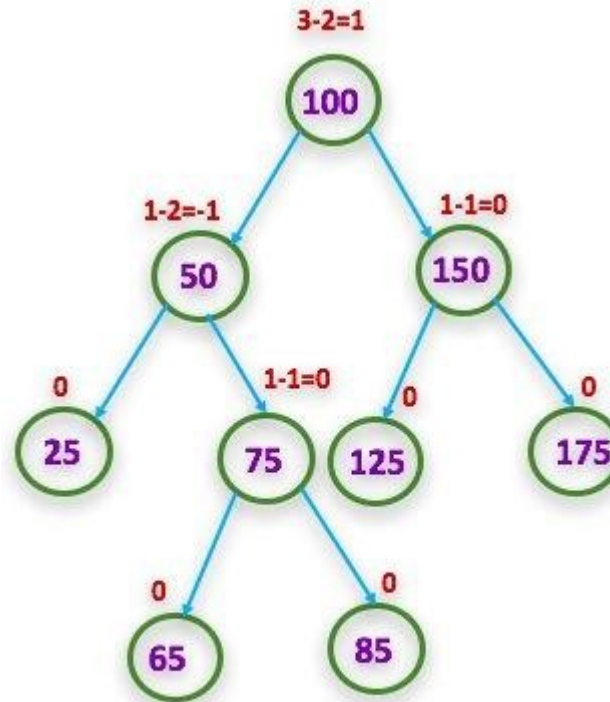
The difference between the heights of the left subtree and the right subtree for any node is known as the **balance factor** of the node.

The AVL tree is named after its inventors, Georgy Adelson-Velsky and Evgenii Landis



AVL tree

Balancing factor



AVL Tree

Balance Factor = (Height of Left Subtree - Height of Right Subtree)

LL Rotation



LR Rotation



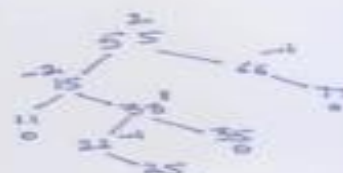
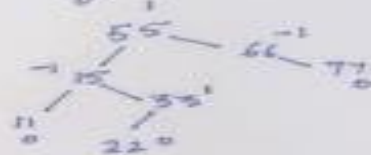
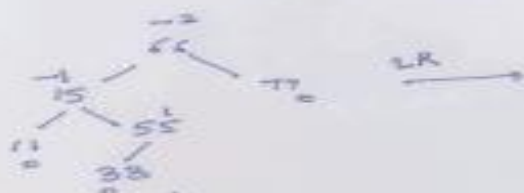
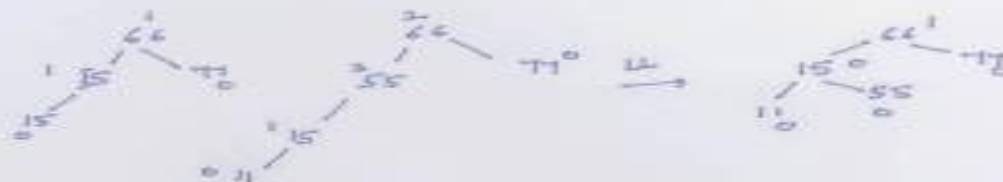
RR Rotation



RL Rotation



55, 66, 77, 15, 11, 33, 22, 35, 25



Q 64, 1, 44, 25, 13, 110, 98, 85



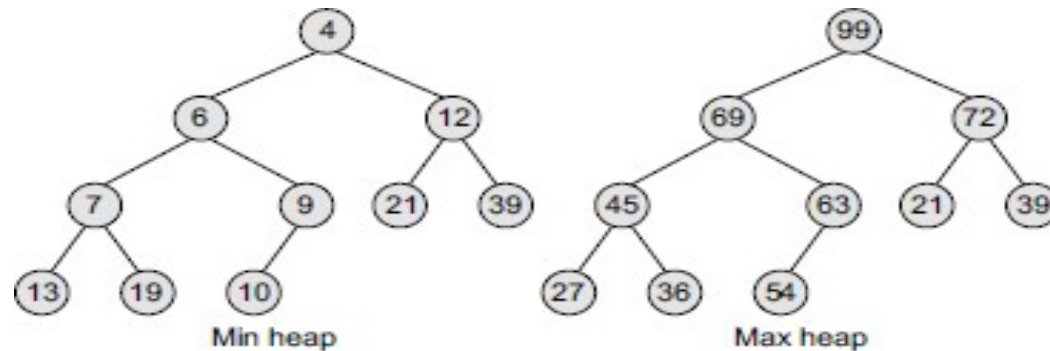
Binary Heap

A binary heap is a complete binary tree in which every node satisfies the heap property which states that:

If B is a child of A, then $\text{key}(A) \geq \text{key}(B)$

This implies that elements at every node will be either greater than or equal to the element at its left and right child. Thus, the root node has the highest key value in the heap. Such a heap is commonly known as a **max-heap**.

Alternatively, elements at every node will be either less than or equal to the element at its left and right child. Thus, the root has the lowest key value. Such a heap is called a **min-heap**



Insertion

Consider a max heap H with n elements. Inserting a new value into the heap is done in the following two steps:

1. Add the new value at the bottom of H in such a way that H is still a complete binary tree but not necessarily a heap.
2. Let the new value rise to its appropriate place in H so that H now becomes a heap as well.

Example 12.2 Build a max heap H from the given set of numbers: 45, 36, 54, 27, 63, 72, 61, and 18. Also draw the memory representation of the heap.

Solution

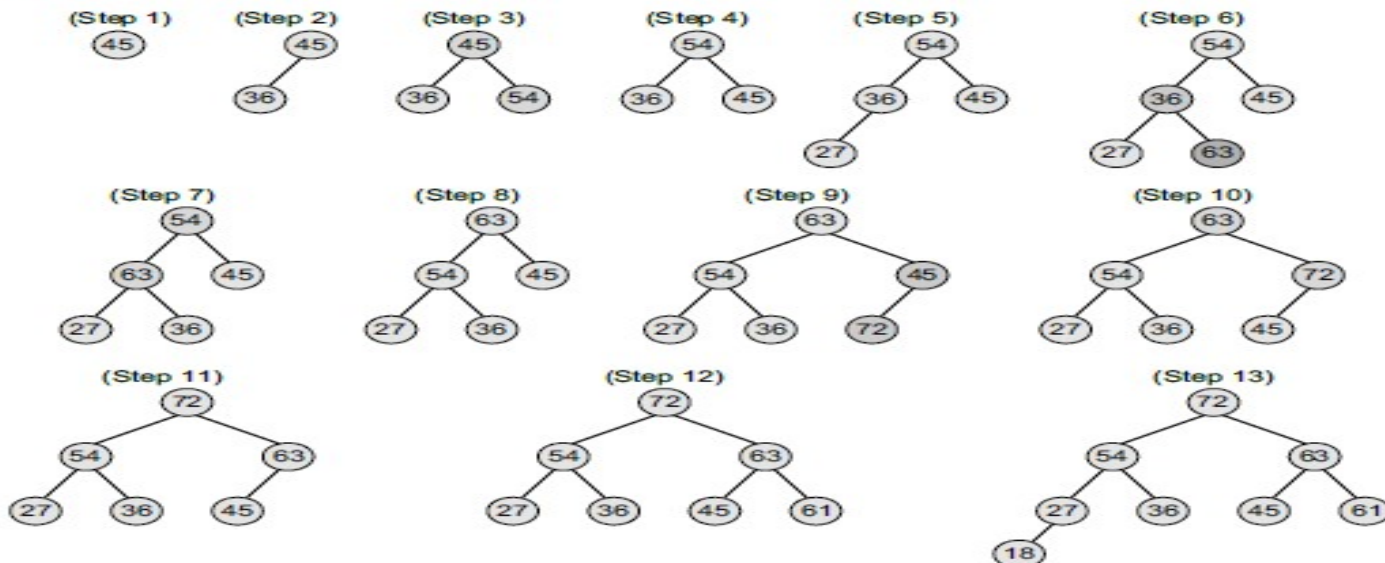


Figure 12.5

The memory representation of H can be given as shown in Fig. 12.6.

HEAP[1]	HEAP[2]	HEAP[3]	HEAP[4]	HEAP[5]	HEAP[6]	HEAP[7]	HEAP[8]	HEAP[9]	HEAP[10]
72	54	63	27	36	45	61	18		

Figure 12.6 Memory representation of binary heap H

Insertion

```

Step 1: [Add the new value and set its POS]
        SET N = N + 1, POS = N
Step 2: SET HEAP[N] = VAL
Step 3: [Find appropriate location of VAL]
        Repeat Steps 4 and 5 while POS > 1
Step 4:     SET PAR = POS/2
Step 5:     IF HEAP[POS] <= HEAP[PAR],
            then Goto Step 6.
            ELSE
                SWAP HEAP[POS], HEAP[PAR]
                POS = PAR
            [END OF IF]
        [END OF LOOP]
Step 6: RETURN
    
```

insertion of a single value may take $O(\log n)$ time, to build a heap of n elements, the algorithm will execute in $O(n \log n)$ time.

Deleting an Element from a Binary Heap

Consider a max heap H having n elements. An element is always deleted from the root of the heap. So, deleting an element from the heap is done in the following three steps:

1. Replace the root node's value with the last node's value so that H is still a complete binary tree but not necessarily a heap.
2. Delete the last node.
3. Sink down the new root node's value so that H satisfies the heap property. In this step, interchange the root node's value with its child node's value (whichever is largest among its children).

```

Step 1: [Remove the last node from the heap]
        SET LAST = HEAP[N], SET N = N - 1
Step 2: [Initialization]
        SET PTR = 1, LEFT = 2, RIGHT = 3
Step 3: SET HEAP[PTR] = LAST
Step 4: Repeat Steps 5 to 7 while LEFT <= N
Step 5: IF HEAP[PTR] >= HEAP[LEFT] AND
        HEAP[PTR] >= HEAP[RIGHT]
        Go to Step 8
        [END OF IF]
Step 6: IF HEAP[RIGHT] <= HEAP[LEFT]
        SWAP HEAP[PTR], HEAP[LEFT]
        SET PTR = LEFT
    ELSE
        SWAP HEAP[PTR], HEAP[RIGHT]
        SET PTR = RIGHT
    [END OF IF]
Step 7: SET LEFT = 2 * PTR and RIGHT = LEFT + 1
        [END OF LOOP]
Step 8: RETURN
    
```

Figure 12.10 Algorithm to delete the root element from a max heap

Example 12.3 Consider the max heap H shown in Fig. 12.8 and delete the root node's value.

Solution

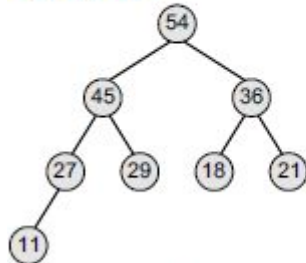
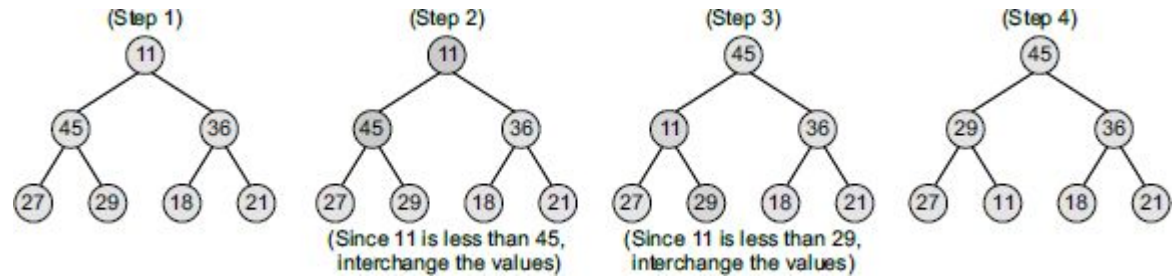


Figure 12.8 Binary heap



Applications of Binary Heaps

Sorting an array using *heapsort* algorithm.

- Build a heap from the given input array.
- Repeat the following steps until the heap contains only one element:
 - Swap the root element of the heap (which is the largest element) with the last element of the heap.
 - Remove the last element of the heap (which is now in the correct position).
 - Heapify the remaining elements of the heap.
- The sorted array is obtained by reversing the order of the elements in the input array.

Implementing priority queues.

Multi-way Search Tree (m-way search tree)

M-way search tree which has $M - 1$ values per node and M subtrees.

In such a tree, M is called the degree of the tree. In other words, every internal node of an M -way search tree consists of pointers to M sub-trees and contains $M - 1$ keys, where

M

> 2 .

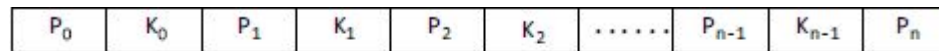


Figure 11.2 Structure of an M -way search tree node

In the structure shown, $P_0, P_1, P_2, \dots, P_n$ are pointers to the node's sub-trees and $K_0, K_1, K_2, \dots, K_{n-1}$ are the key values of the node. All the key values are stored in ascending order. That is, $K_i < K_{i+1}$ for $0 \leq i \leq n-2$.

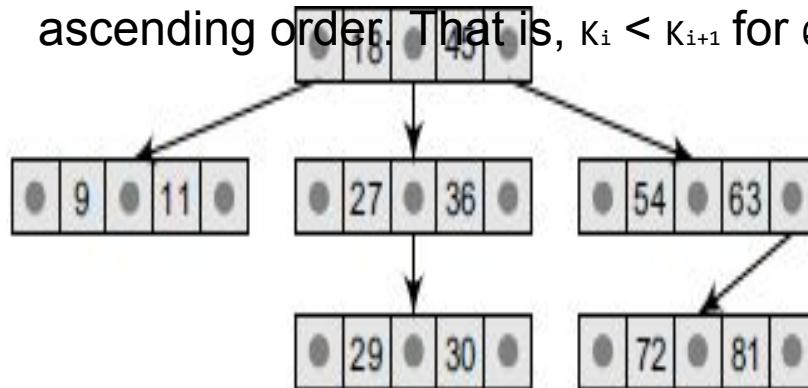


Figure 11.3 M -way search tree of order 3

Properties of an M -way search tree.

- All the key values in the sub-tree pointed by P_i are less than K_i , where $0 \leq i \leq n-1$.
- All the key values in the sub-tree pointed by P_i are greater than K_{i-1} , where $0 \leq i \leq n-1$.

B-Tree

A B tree is a specialized M-way tree developed by Rudolf Bayer and ^SEd McCreight in 1970 that is widely used for disk access. A B tree of order m can have a maximum of $m-1$ keys and m pointers to its sub-trees.

A B tree of order m (the maximum number of children that each node can have) is a tree with all the properties of an M-way search tree. In addition it has the following properties:

1. Every node in the B tree has at most (maximum) m children.
2. Every node in the B tree except the root node and leaf nodes has at least (minimum) $m/2$ children. This condition helps to keep the tree bushy so that the path from the root node to the leaf is very short, even in a tree that stores a lot of data.
3. The root node has at least two children if it is not a terminal (leaf) node.
4. All leaf nodes are at the same level.

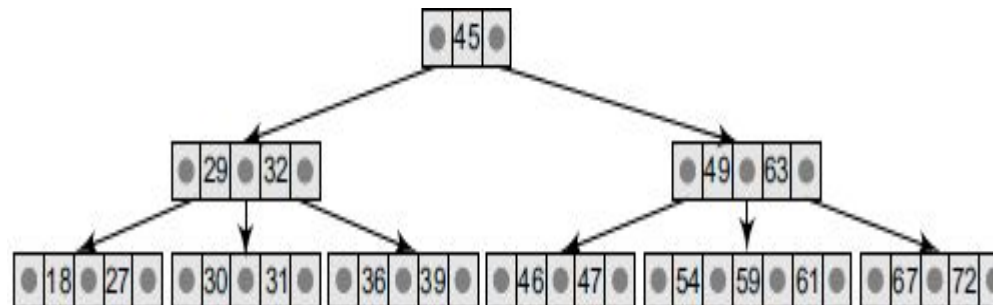


Figure 11.4 B tree of order 4

Operations on B-Tree

Searching for an element in a B tree is similar to that in binary search trees.

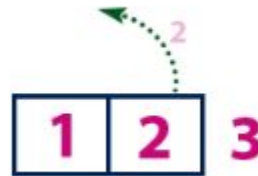
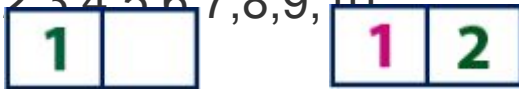
Insertion

In a B tree, all insertions are done at the leaf node level. A new value is inserted in the B tree using the algorithm given below.

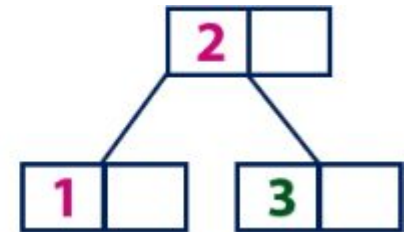
1. Search the B tree to find the leaf node where the new key value should be inserted.
2. If the leaf node is not full, that is, it contains less than $m-1$ key values, then insert the new element in the node keeping the node's elements ordered.
3. If the leaf node is full, that is, the leaf node already contains $m-1$ key values, then
 - (a) insert the new value in order into the existing set of keys,
 - (b) split the node at its median into two nodes (note that the split nodes are half full), and
 - (c) push the median element up to its parent's node. If the parent's node is already full, then split the parent node by following the same steps.

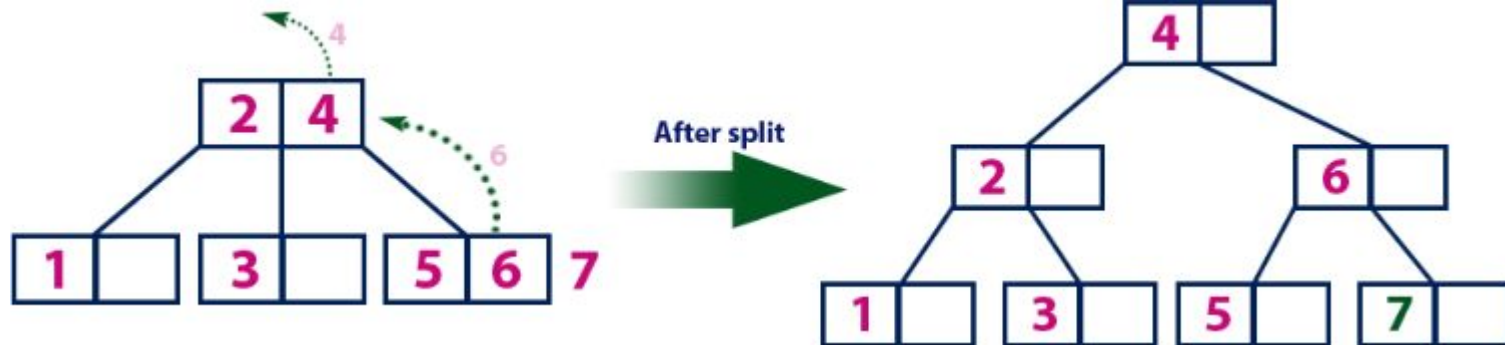
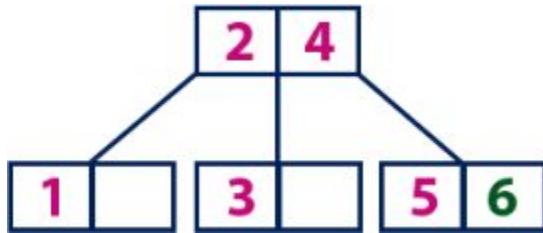
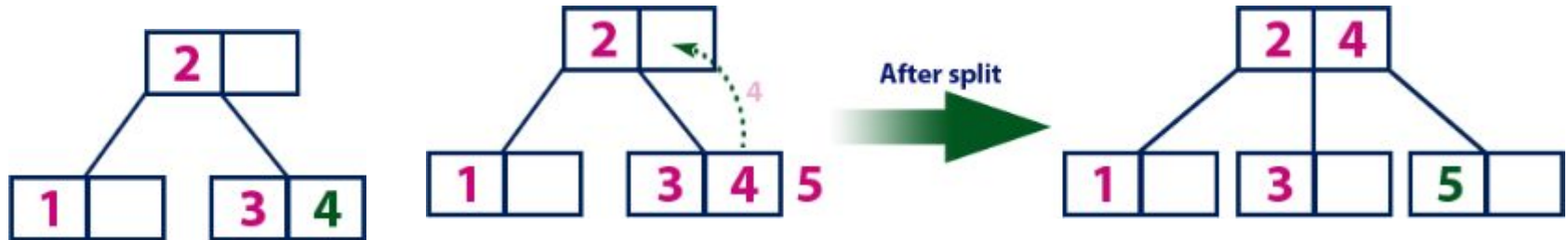
Construct a **B-Tree of Order 3** by inserting numbers from 1 to 10.

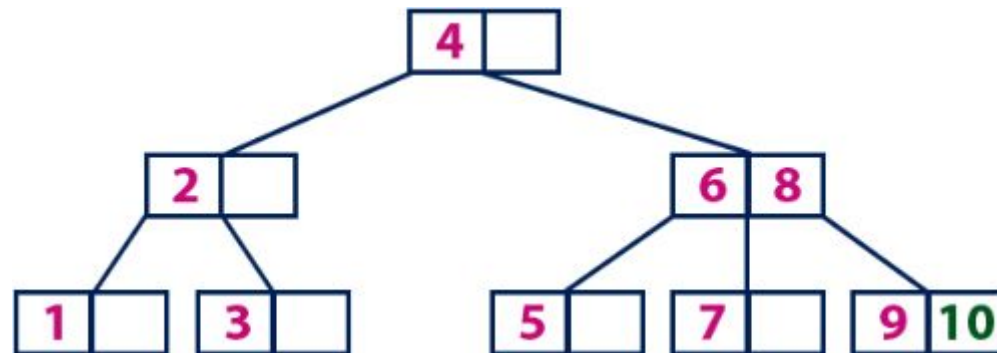
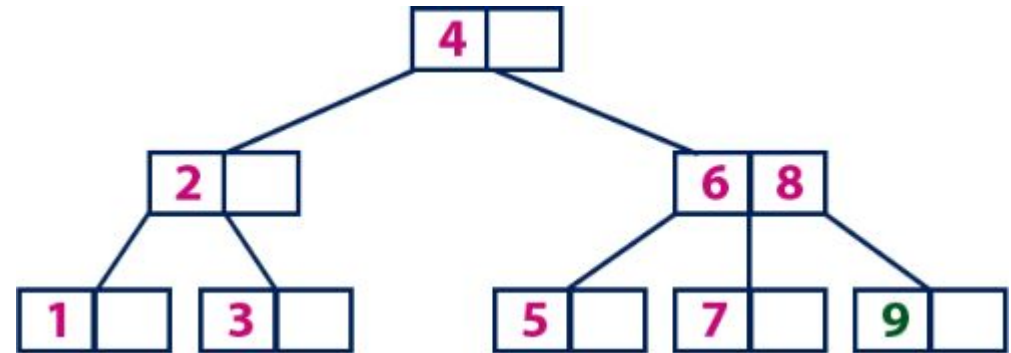
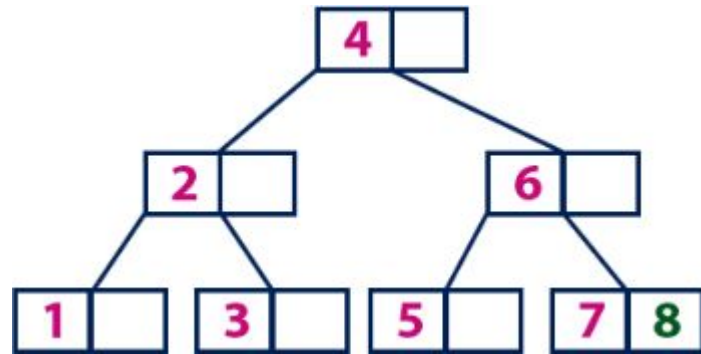
1, 2, 3, 4, 5, 6, 7, 8, 9, 10



After split





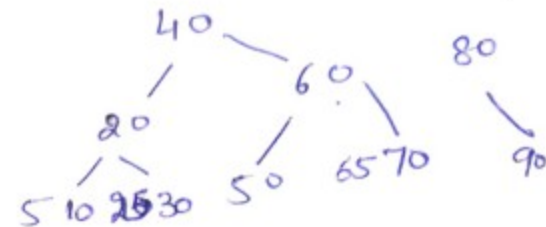
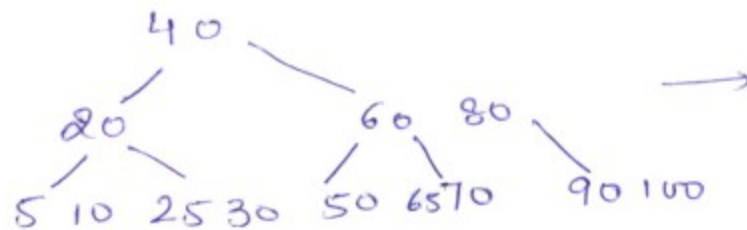


Deletion from B-Tree

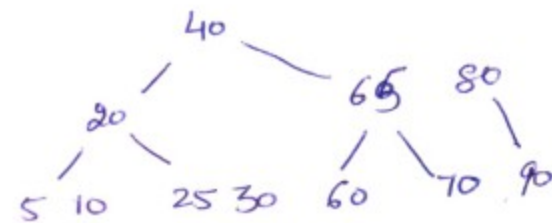
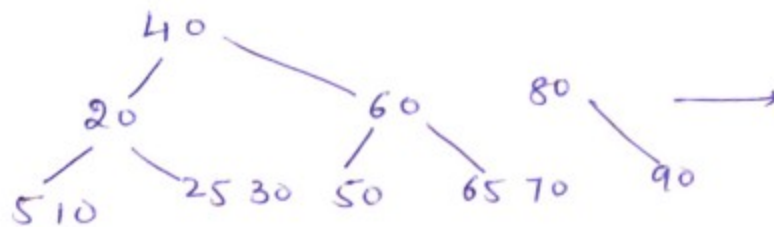
Case I: Delete a leaf node

- A) The deletion of the key does not violate the property of the minimum number of keys a node should hold.
- B) b) The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.
- c) If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**

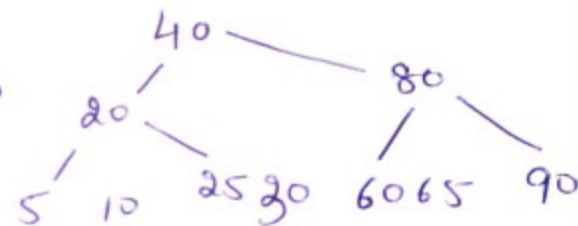
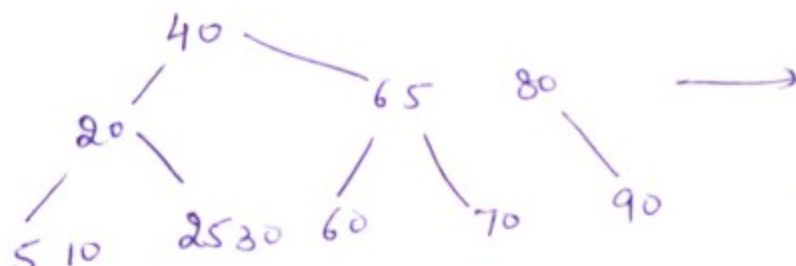
Case 1 a) delete 90



b) delete 50



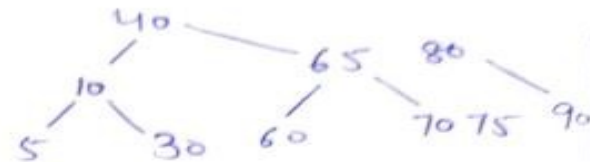
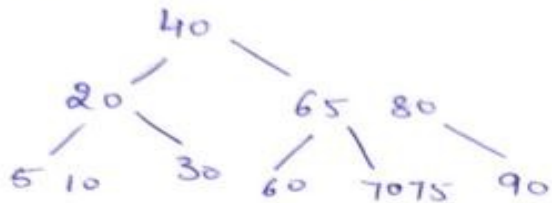
c) delete 70



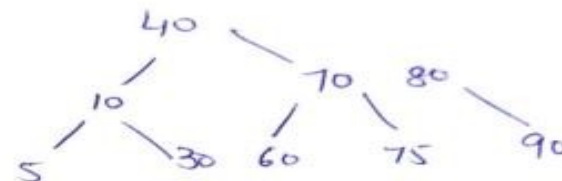
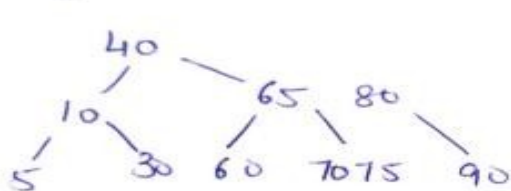
Case II : Delete internal node

1. The internal node, which is deleted, is replaced by an in-order predecessor if the left child has more than the minimum number of keys.
2. The internal node, which is deleted, is replaced by an in-order successor if the right child has more than the minimum number of keys.
3. If either child has exactly a minimum number of keys then merge the left and the right children.

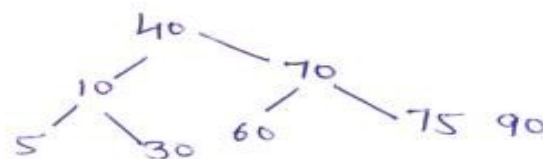
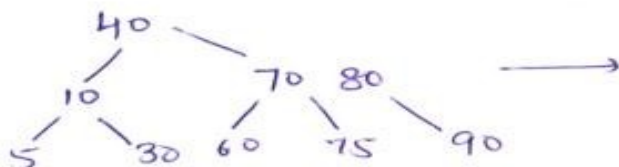
Case 2 a) Delete 20



b) Delete 65



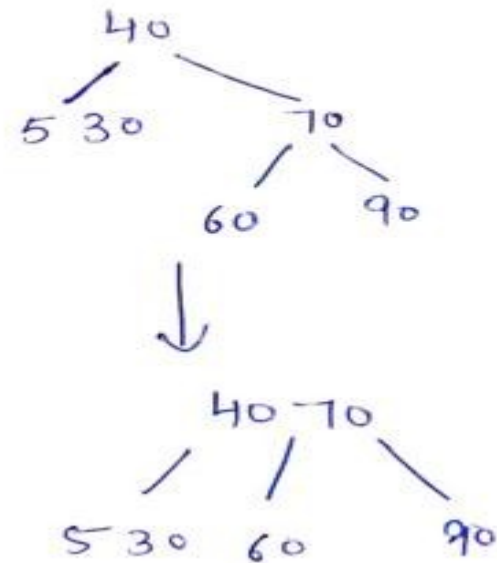
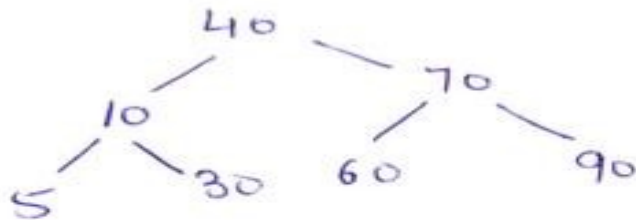
c) Delete 80



Case 3: In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(3) i.e. merging the children.

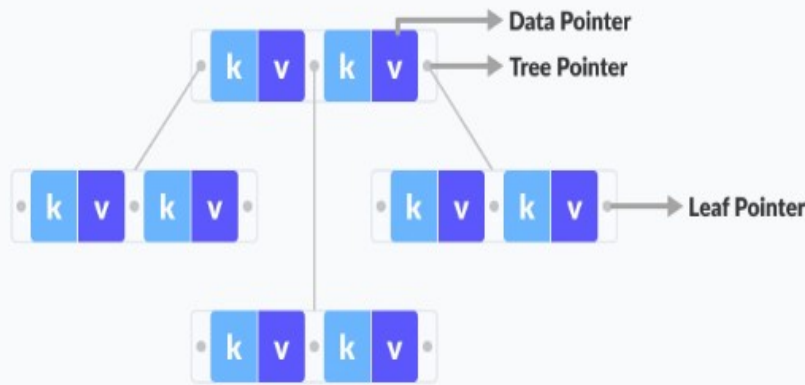
Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).

Case 3 Delete 10

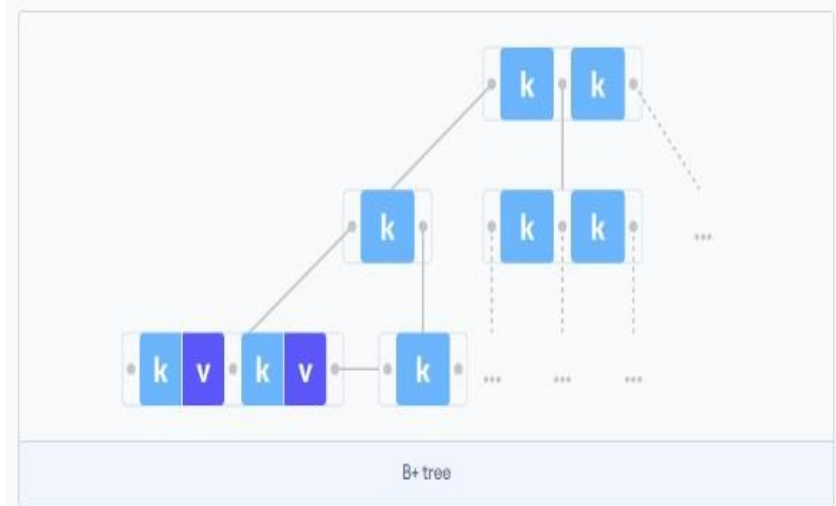


B+ tree

B + Tree is a variation of the [B-tree](#) data structure. In a B + tree, [data pointers](#) are stored only at the leaf nodes of the tree. In a **B+ tree structure** of a leaf node differs from the structure of [internal nodes](#). The leaf nodes have an entry for every value of the search field, along with a data pointer to the record (or to the block that contains this record). The leaf nodes of the B+ tree are linked together to provide ordered access to the search field to the records. Internal nodes of a B+ tree are used to guide the search. Some search field values from the leaf nodes are repeated in the internal nodes of the B+ tree.



B-tree



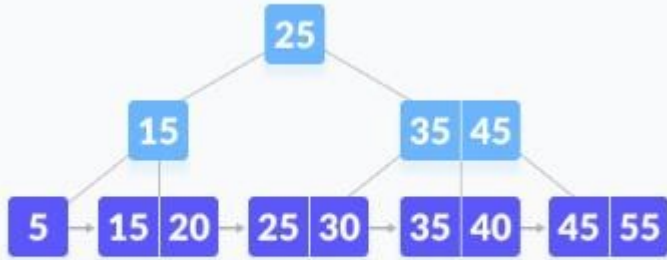
B+ tree

Difference between B-tree and B+tree: The data pointers are present only at the leaf nodes on a B+ tree whereas the data pointers are present in the internal, leaf or root nodes on a B-tree.

The leaves are not connected with each other on a B-tree whereas they are connected on a B+ tree.

Operations on a B+ tree are faster than on a B-tree.

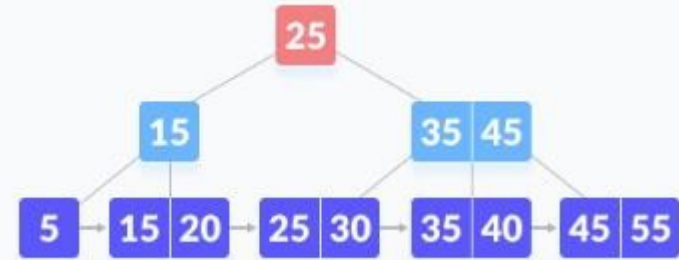
Search 45



B+ tree

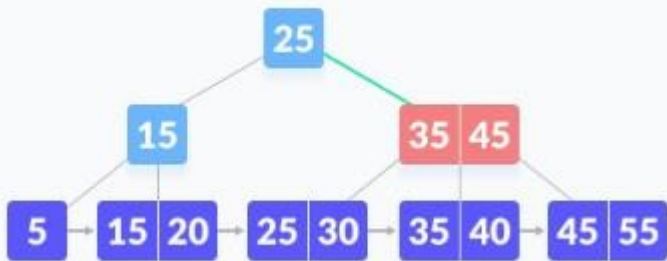
Searching on a B+tree

Compare 45 with root . $45 > 25$ move to right

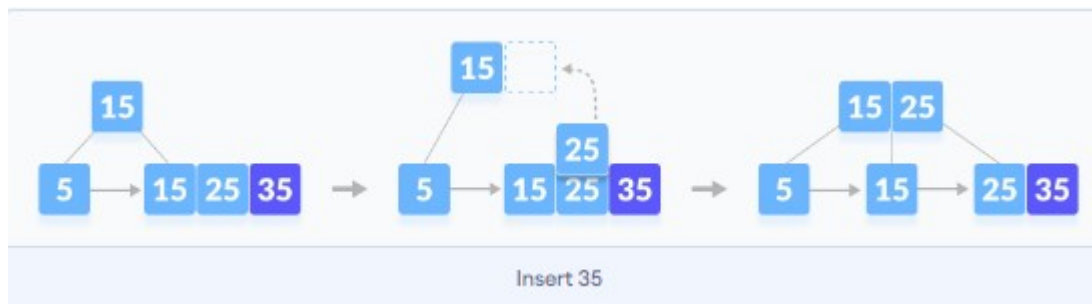


Compare 45 with 35 move to 45.
 $45 \geq 45$ move to right

45 found



Create a 3 way B+ tree
5,15,25,35,45



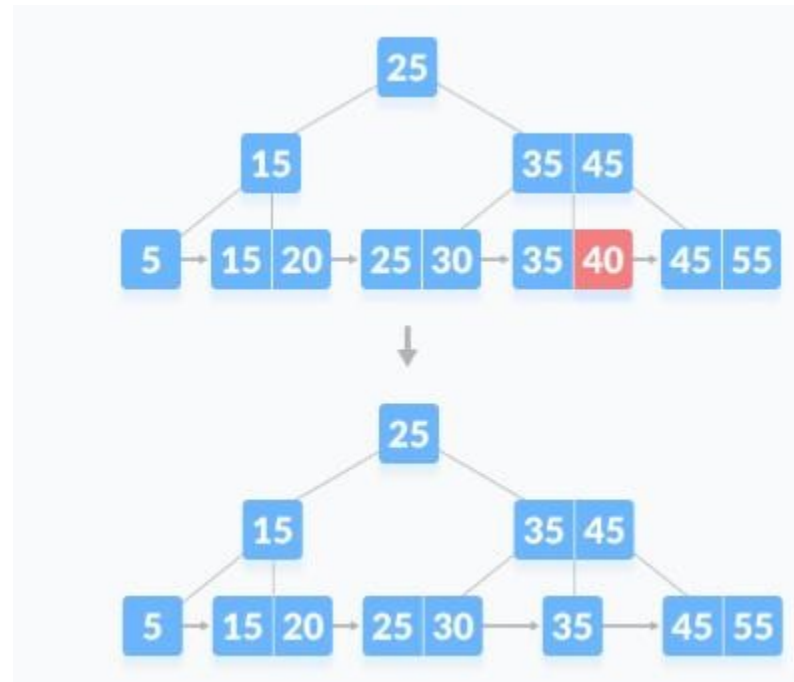


Case I

The key to be deleted is present only at the leaf node not in the indexes (or internal nodes). There are two cases for it:

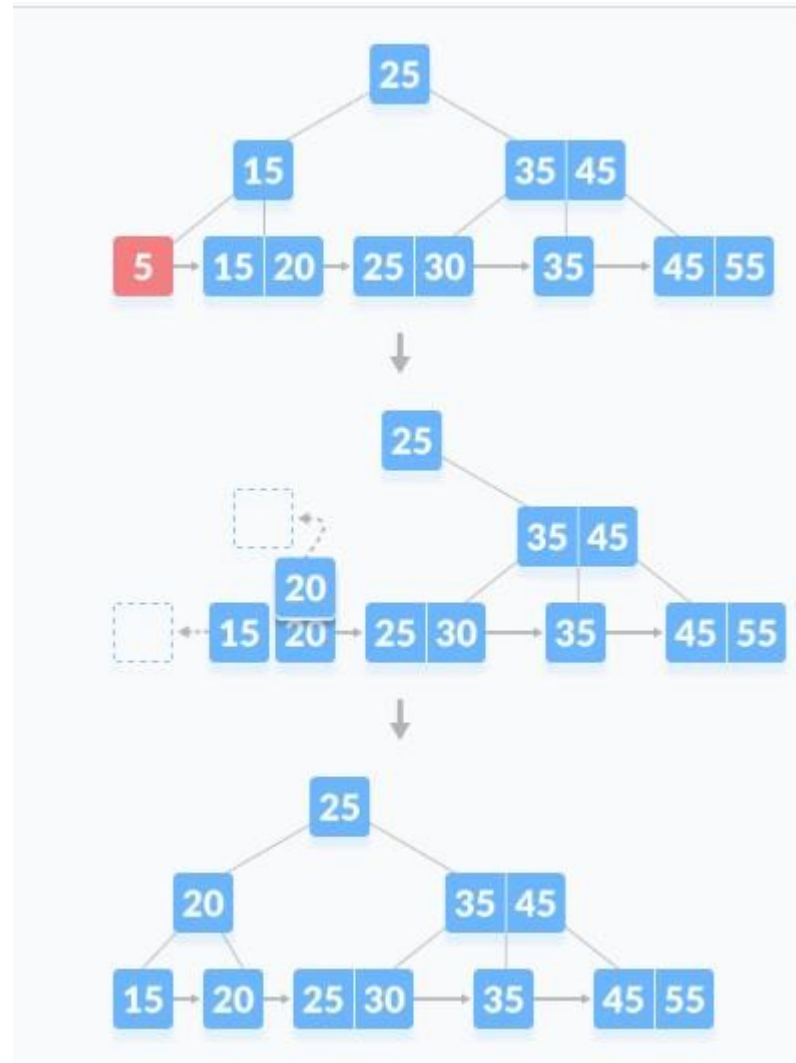
a) There is more than the minimum number of keys in the node. Simply delete the key.

Delete 40



b) There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Add the median key of the sibling node to the parent.

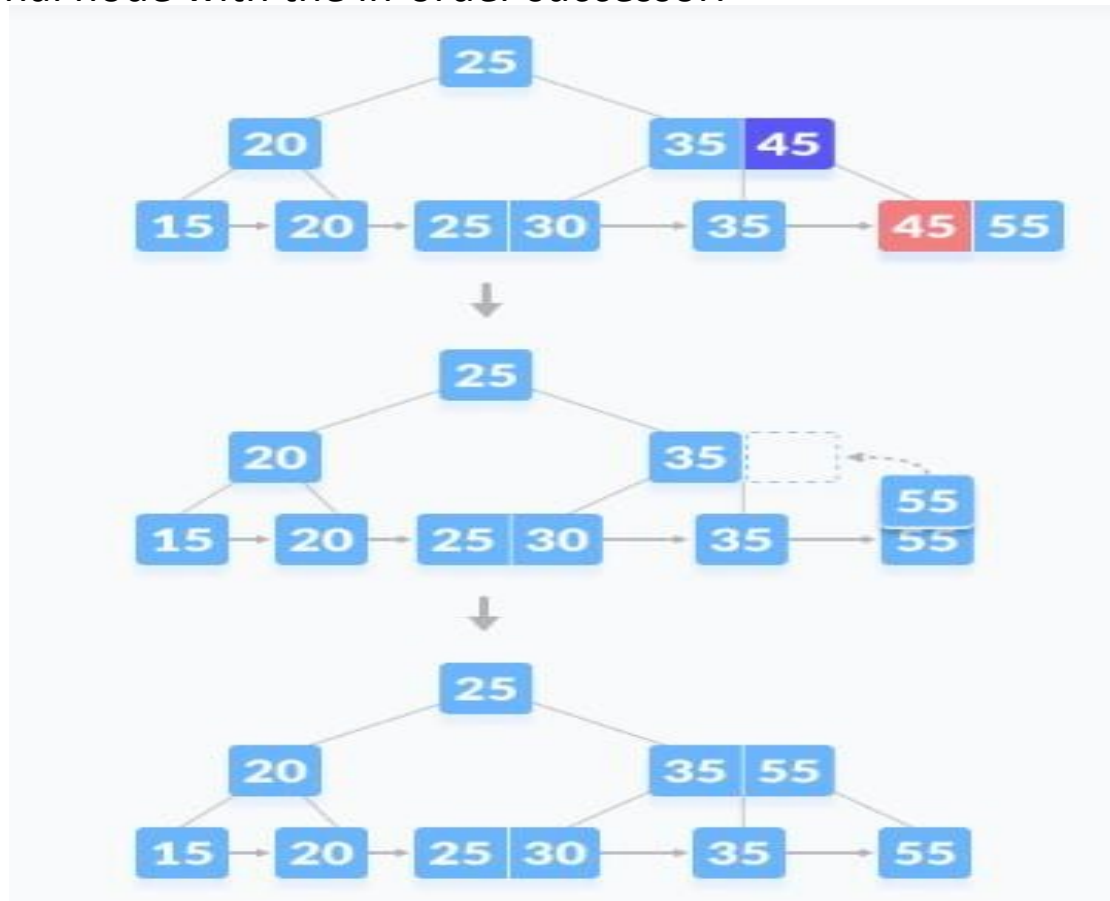
Delete 5



Case II

The key to be deleted is present in the internal nodes as well. Then we have to remove them from the internal nodes as well. There are the following cases for this situation.

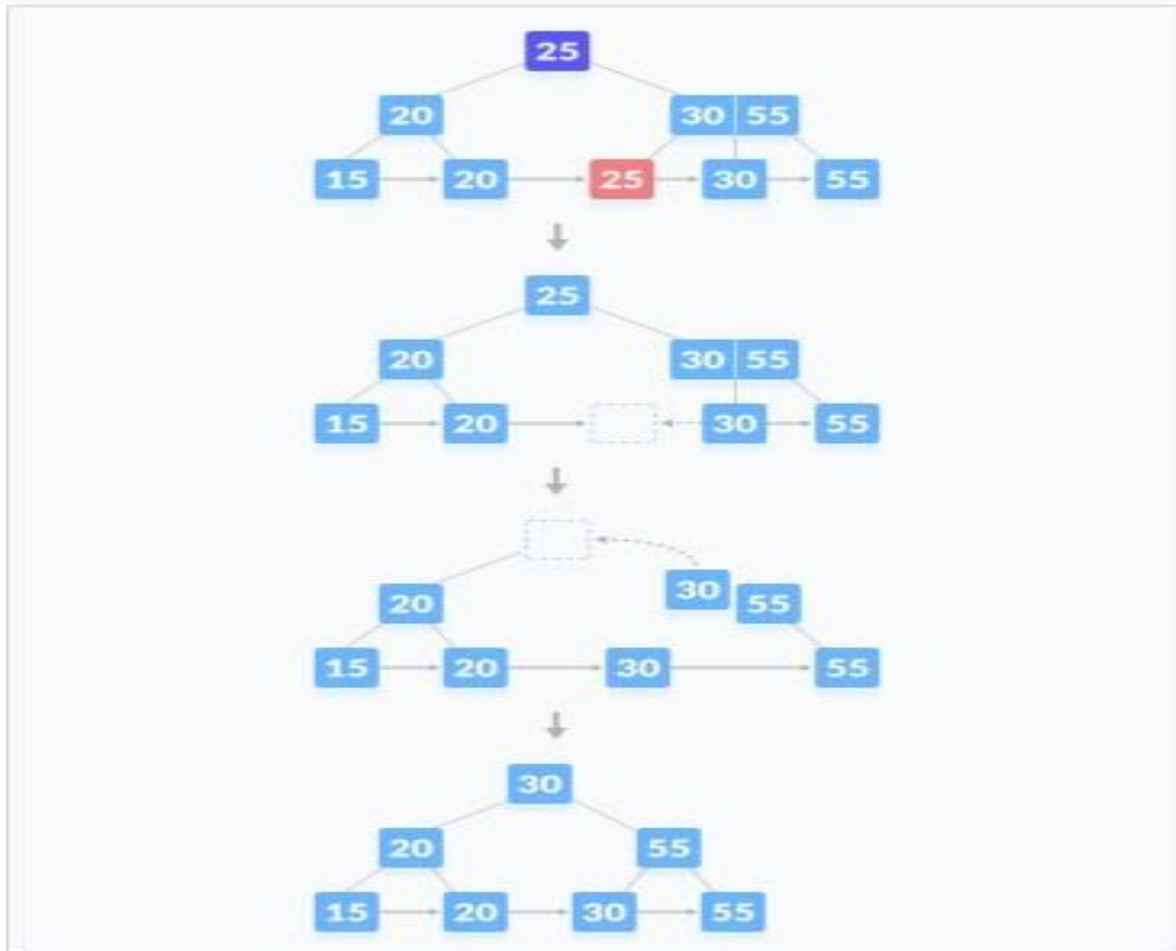
a) If there is more than the minimum number of keys in the node, simply delete the key from the leaf node and delete the key from the internal node as well. Fill the empty space in the internal node with the in-order successor.



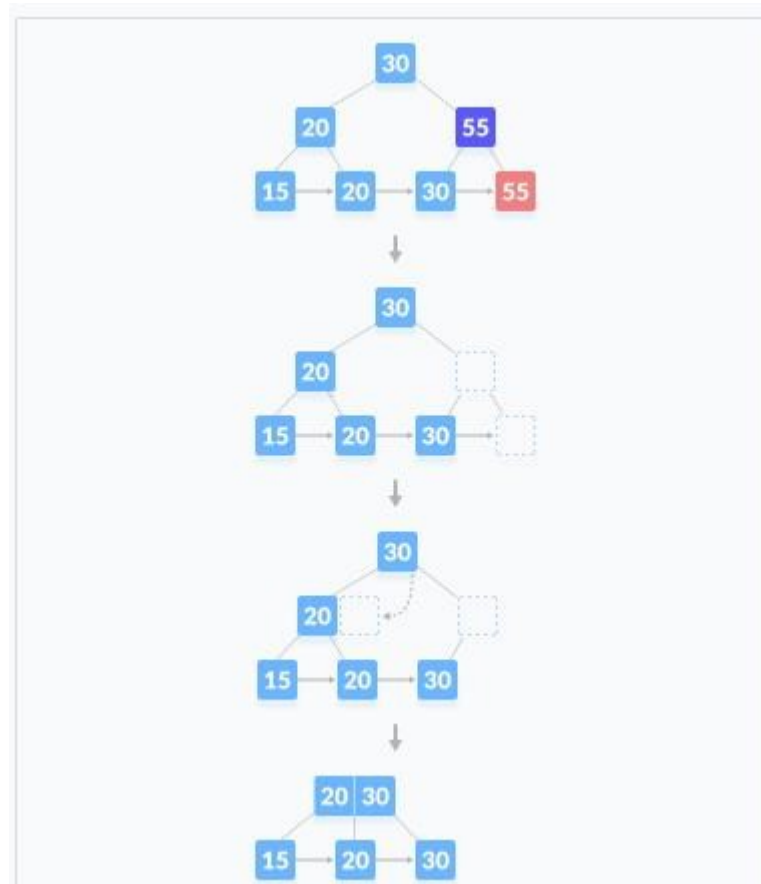
b) If there is an exact minimum number of keys in the node, then delete the key and borrow a key from its immediate sibling (through the parent). Fill the empty space created in the index (internal node) with the borrowed key.



c) This case is similar to Case II(1) but here, empty space is generated above the immediate parent node. After deleting the key, merge the empty space with its sibling. Fill the empty space in the grandparent node with the in-order successor.

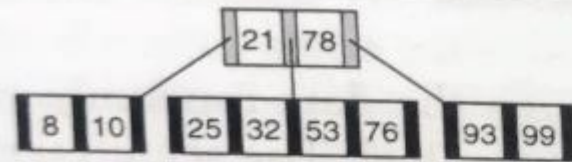


Case III In this case, the height of the tree gets shrunk. Deleting 55 from the tree below leads to this condition.

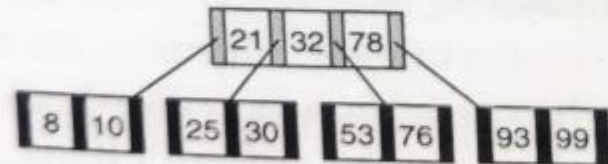


B* tree

- Each node contains minimum of $2/3$ nodes rather than half
- In B* tree, when a node overflows instead of being spilt immediately, the data are redistributed among the nodes siblings, delaying the creation of new node.



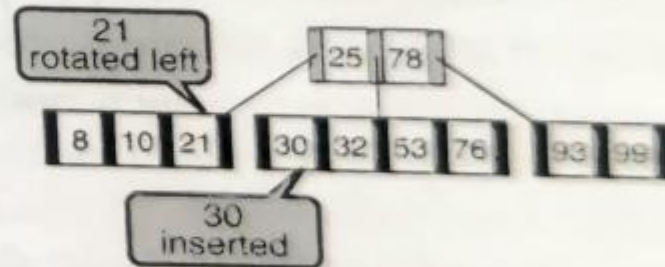
Before adding 30



(a) Insertion into B-tree of order 5



Before adding 30



Result after redistribution
(b) Insertion into B*-tree of order 5

B*tree Insertion

END